

PDF-Export und Druck

Funktionsweise in der ClupData-Anwendung

1. Architektur-Übersicht

Das PDF-Export-System folgt einem **Template-Pattern** mit strenger Entkopplung zwischen UI, Datenextraktion und PDF-Generierung. Der folgende Graph zeigt den vollständigen Datenfluss vom Auslösen des Exports bis zur Ausgabe (Drucken/Speichern):

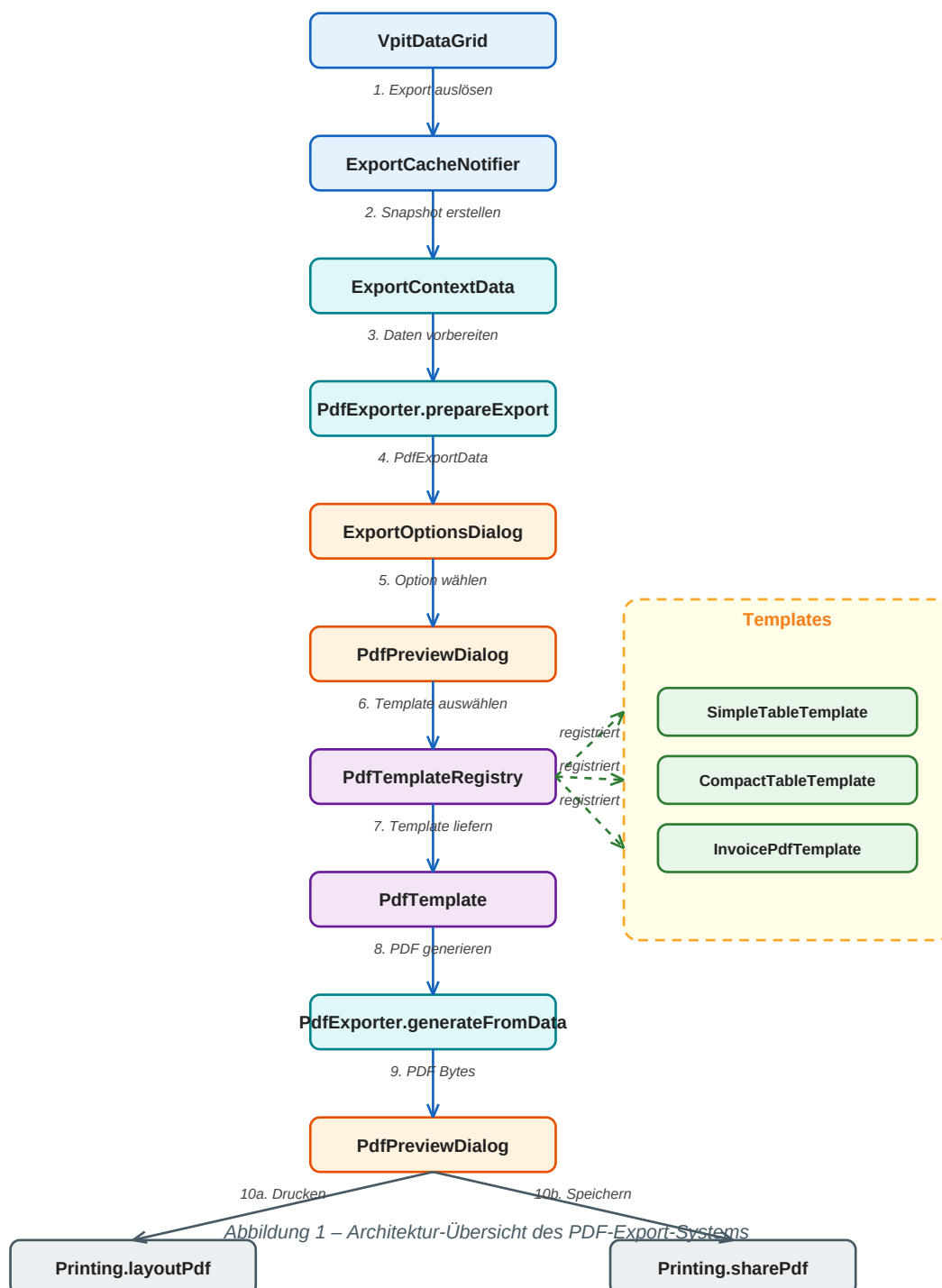


Abbildung 1 – Architektur-Übersicht des PDF-Export-Systems

2. Beteiligte Klassen und ihre Rollen

2.1 Datenextraktion & Kontext

Klasse	Datei	Rolle
ExportContextData	export_context_provider.dart	Reiner OOP-Datencontainer mit Snapshot: Modus (Liste/Detail), DataTable, F
ExportDataTable	export_data_table.dart	Generisches DTO für tabellarische Daten mit Headern und vorformatierten Ze
ExportConfig	export_config.dart	Konfiguration für Export: Entity-Typ, Titel, optionales Item
ExportCacheNotifier	export_context_provider.dart	Riverpod-Notifier mit Stack von Generator-Funktionen für aktuelle UI-Snapsho

2.2 PDF-Generierung

Klasse	Datei	Rolle
PdfExporter	pdf_exporter.dart	Orchestriert PDF-Export: prepareExport() und generateFromData()
PdfExportData	pdf_exporter.dart	Datencontainer für PDF-Generierung mit DataTable, Context, Entity-Typ
PdfExportContext	pdf_export_context.dart	Metadaten: Titel, Zeitstempel, aktive Filter/Sortierung, Detail-Flag

2.3 Template-System

Klasse	Datei	Rolle
PdfTemplate	pdf_template.dart	Interface: generate(), supportsDetailView, supportedEntityTypes, category
PdfTemplateRegistry	pdf_template_registry.dart	Zentrale Registry: register(), get(), getSuitableFor(), getDefaultFor()
SimpleTableTemplate	simple_table_template.dart	Default-Template: Generische Tabelle mit Headern, alternierenden Farben
CompactTableTemplate	compact_table_template.dart	Kompaktes Tabellenlayout für Übersichten

2.4 UI-Komponenten

Klasse	Datei	Rolle
PdfPreviewDialog	pdf_preview_dialog.dart	Vorschau-Dialog: Template-Auswahl, Live-PDF-Vorschau, Drucken/Speich
ExportOptionsDialog	export_options_dialog.dart	Optionen-Dialog: Wahl zwischen aktueller Ansicht, allen Details, vollem Ex
PdfTemplateSelector	pdf_template_selector.dart	UI für Template-Auswahl mit Kategorisierung

2.5 Externe Bibliotheken

Bibliothek	Verwendung
pdf (package:pdf)	PDF-Dokumentenerstellung mit pw.Document, pw.MultiPage, Tabellen
printing (package:printing)	System-Druckdialog (Printing.layoutPdf) und Teilen (Printing.sharePdf)

3. Ablauf im Detail

Schritt 1: Export auslösen

Der Export wird typischerweise über das **VpitDataGrid** Toolbar-Menü ausgelöst. Das Grid hat eine *exportConfig*-Property, die den Export-Menü-Button rendert.

```
ListExportMenuButton<T>(  
    controller: controller,  
    exportConfig: exportConfig,  
)
```

Schritt 2: Snapshot erstellen

Der **ExportCacheNotifier** hält einen Stack von Generator-Funktionen. Diese werden vom VpitDataGrid registriert und erstellen synchron einen Snapshot des aktuellen UI-Zustands:

```
ref.read(exportCacheNotifierProvider.notifier).pushGenerator(  
    () => ExportContextData(  
        mode: ExportMode.list,  
        dataTable: currentDataTable,  
        fullDataTable: fullDataTable,  
        activeFilters: currentFilters,  
        activeSorts: currentSorts,  
        entityType: 'mitglied',  
        title: 'Mitgliederliste',  
    ),  
);
```

Schritt 3: Export-Optionen wählen

Der **ExportOptionsDialog** zeigt drei Optionen: (1) *Aktuelle Ansicht* – nur sichtbare Spalten und gefilterte Daten, (2) *Alle Details* – alle Spalten inkl. ausgeblendeter, (3) *Voller Export* – mit Relationsdaten (falls verfügbar).

Schritt 4: PDF vorbereiten

Der **PdfExporter** bereitet die Export-Daten vor:

```
PdfExportData prepareExport(ExportContextData contextData,  
    {bool useFullTable = false}) {  
    final table = (useFullTable && contextData.fullDataTable != null)  
        ? contextData.fullDataTable!  
        : contextData.dataTable;  
    final pdfContext = PdfExportContext(  
        title: contextData.title,  
        exportTimestamp: DateTime.now(),  
        activeFilters: contextData.activeFilters,  
        activeSorts: contextData.activeSorts,  
        isDetailView: contextData.isDetail,  
        entityName: contextData.entityType,  
    );  
    return PdfExportData(  
        dataTable: table,  
        context: pdfContext,  
        isDetailView: contextData.isDetail,  
        entityType: contextData.entityType,
```

```
        detectedEntityType: _detectEntityType(contextData.entityType),  
    );  
}
```